FLOW • SOURCE → DECISION → EXECUTION

从来源到执行
数据与决策流

来源只把外部事实变成 Trigger；Intake 去重与扇出；Evals 与主 Session 决定接不接；Tickets 用 Mandate 授权；DshExecutor 派发 worker Session；Replies 把结果写回来源。14 张图 + 逐字段的 UI 设置映射。

Kanban 看板 | Gitea 宪章 | ntfy 主题 | MinIO 资产 | 14 diagrams | settings → code

文档	相关	生成方式
FLOW.md	**BLK.md • docs/005 • docs/007**	**node scripts/build-flow-doc.mjs**

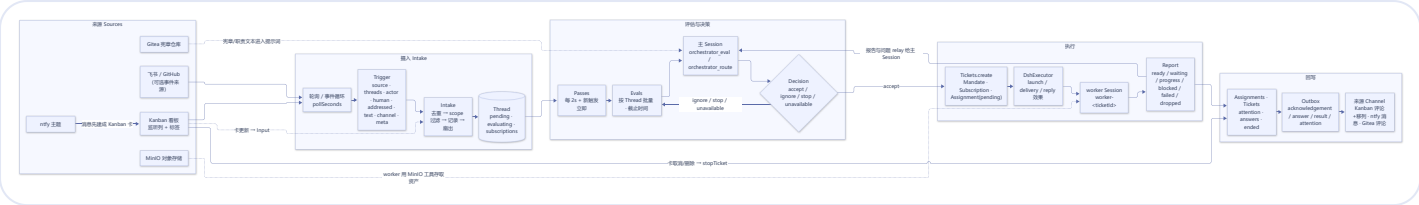
1. 总览
2. 来源层：数据怎么进来
3. 摄入 Intake：去重、过滤、扇出
4. 评估 Evals：批量、语义强度、截止时间
5. 决策：接不接、以什么强度接
6. 接单：工单、授权、派工
7. 看板卡的全链路时序（显式请求）
8. 执行内部（DshExecutor）
9. 失败、等待与恢复
10. 数据模型
11. 关键不变量与常见误解
12. 从代码读流程的入口索引
13. UI 设置 → 存储 → 代码 → 运行时：映射与关联

FLOW • 从来源到执行的数据与决策流

> 本文回答三个问题：**数据从哪来**（来源 → Trigger）、**谁在什么条件下接单**（准入 → 评估 → 决策）、**接单后怎么执行与回写**（工单 → 派工 → worker → 报告 → 回复）。 阅读版：FLOW.html · FLOW.pdf（AIOS 同款深色主题，图已预渲染）。 配套阅读：docs/005-panel.md（来源配置）、docs/007-architecture.md（模块职责）、 docs/003-execution.md（工单阶段表）。

---

1. 总览



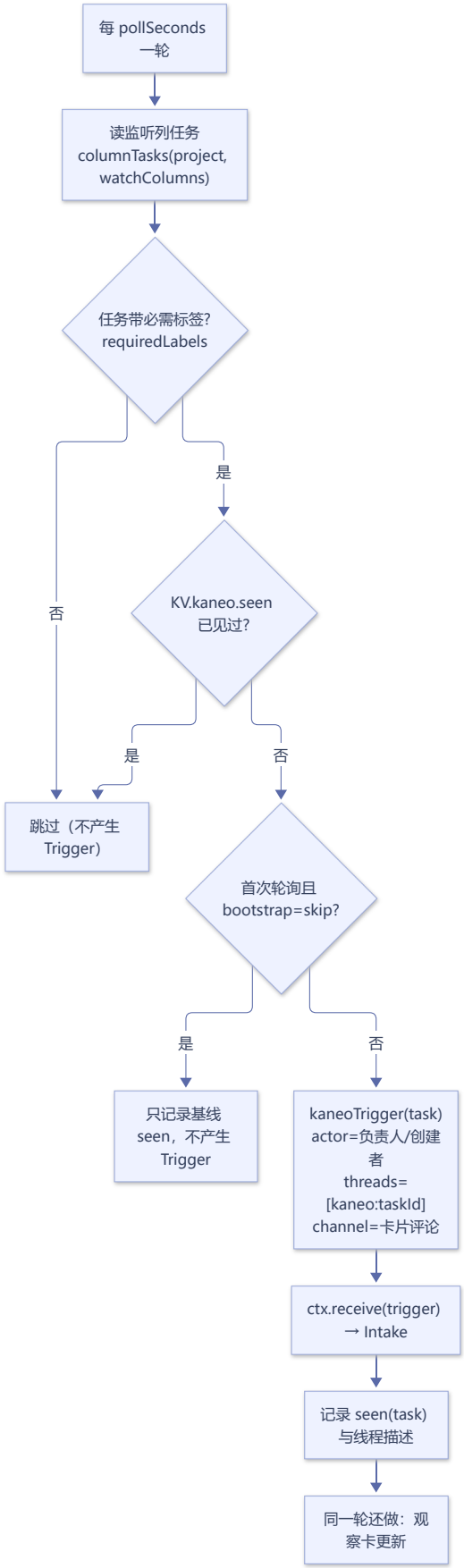
一句话概括：**来源只负责把外部事实变成 Trigger；Intake 负责去重与扇出；Evals + 主 Session 负责“接不接”；Tickets 负责“以什么授权接”；DshExecutor 负责“谁、在哪个目录、用什么 Session 去执行”；Replies 负责“结果怎么回去”。**

---

2. 来源层：数据怎么进来

来源	角色	进入系统的形态	是否直接产生任务
Kanban 看板	唯一的直接任务来源	监听列中的任务 → `Trigger`（`human=true`，`addressed=direct`）	✅
ntfy 主题	消息入口	新消息先**建成 Kanban 卡**（to-do 池），由看板调度	❌（间接）
Gitea（宪章仓库）	文档/宪章来源	宪章与 `duties/` 文本进入评估与 worker 提示词	❌
MinIO 对象存储	资产/交付物载体	worker 通过 MinIO 工具读写；不产生任务	❌
飞书 / GitHub	可选的事件来源	消息 / Issue / PR → Trigger（与 Kanban 同一管道）	✅

2.1 Kanban 轮询



- `bootstrap=skip`：第一次看到的历史卡只登记、不接单；`bootstrap=trigger` 则把已有卡也当请求。
- `KV.kaneo.seen`：同一任务只产生一次 Trigger，之后靠“卡更新观察”产生增量 Input。
- **看板观察**（`observeUpdates`）：运行中工单的卡出现**评论/标题/描述/列**变化 → 生成更新 Trigger（`human=true`，`meta.update=true`）→ 作为 Input 送给同一工单的 Session，**不新开工单**。

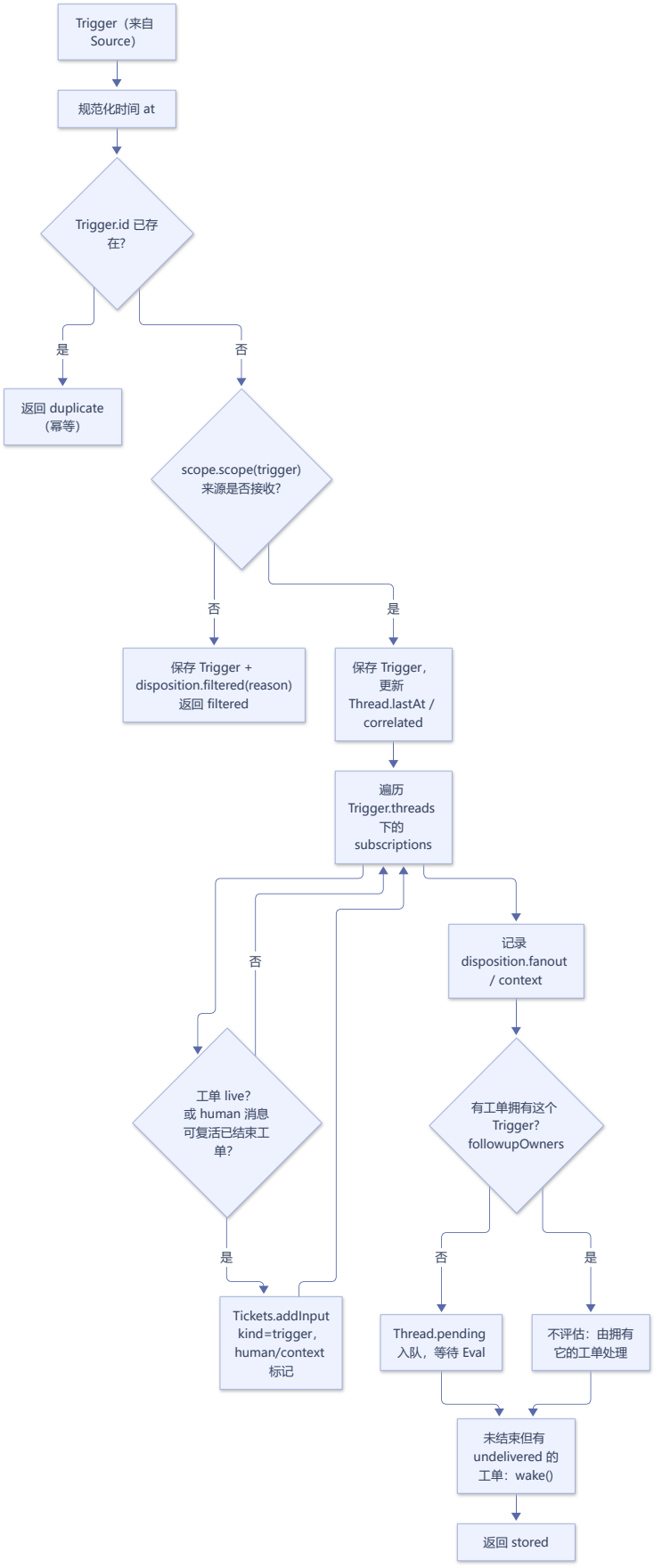
- **看板不裁决、不取消**：卡上的状态只作为更新来源；工单要停只能由人在面板点「停止」，卡上的标签/列变化不会停工单（此前的「hold 标签恢复 / 取消列停止」设计已取消）。

2.2 其他来源

- **ntfy**：轮询主题历史 → 每条消息调用 Kanban API 建卡（`to-do` 列）；卡进入看板后才走 Kanban 管道。
- **Gitea**：面板“测试连接”与实际健康检查都只校验 `charterRepository`；它不监听 Issue/PR，卡片的 `repo:owner/name` 才决定工单用哪个代码仓库。
- **MinIO**：`s3` 来源只做连通/桶探测与工具能力，不产生 Trigger。

---

3. 摄入 Intake：去重、过滤、扇出



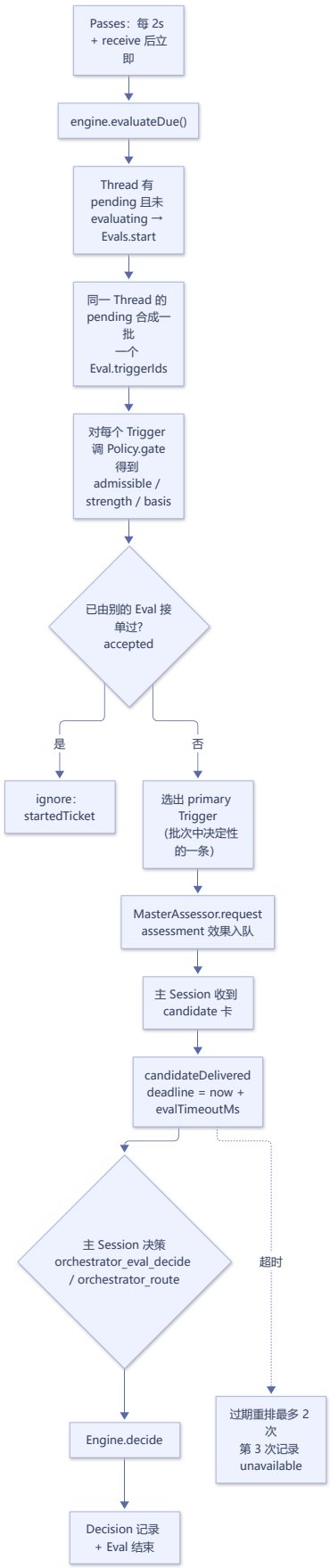
要点：

1. **先过滤再落库**：被来源拒绝的触发也会保存，并带 `filtered.reason`（面板“被过滤”可查）。
2. **fan-out 到订阅该 Thread 的工单**：人写的消息送到（live 工单，或非 human-stop 的已结束工单——后者进入“可复活”队列）。

3. **没有工单拥有时才评估**：`owners(trigger)` 为空才 `queueForEval`；否则它只是该工单的 follow-up Input。
4. **human 消息可复活已结束工单**：`Tickets.wake` 在下一轮 `evaluateDue` 中按当前 Policy 重新验证 Mandate；不通过则 `refuse`。

---

4. 评估 Evals：批量、语义强度、截止时间



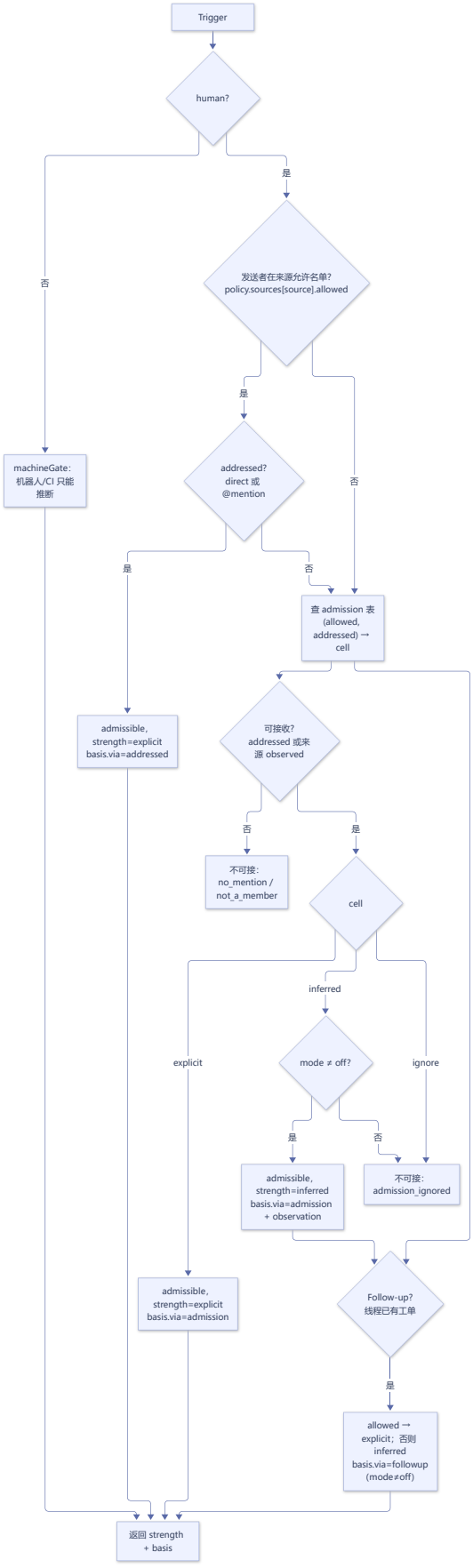
- **一个 Thread 一个 Eval**：批处理让“同一线程里连续几条消息”只产生一次研判。
- **截止时间从 candidate 送达算起**；`paused` 时暂停过期，恢复后 deadline 顺延。
- **过期**：重新排队（`REQUEUES=2`），第三次记为 `unavailable`（面板显示为问题，可 dismiss）。

- 主 Session 拿到的是**候选卡**：线程上下文（`threadContext`，含仓库/线程读取边界）、批次内其他触发、匹配规则与授权信息。

---

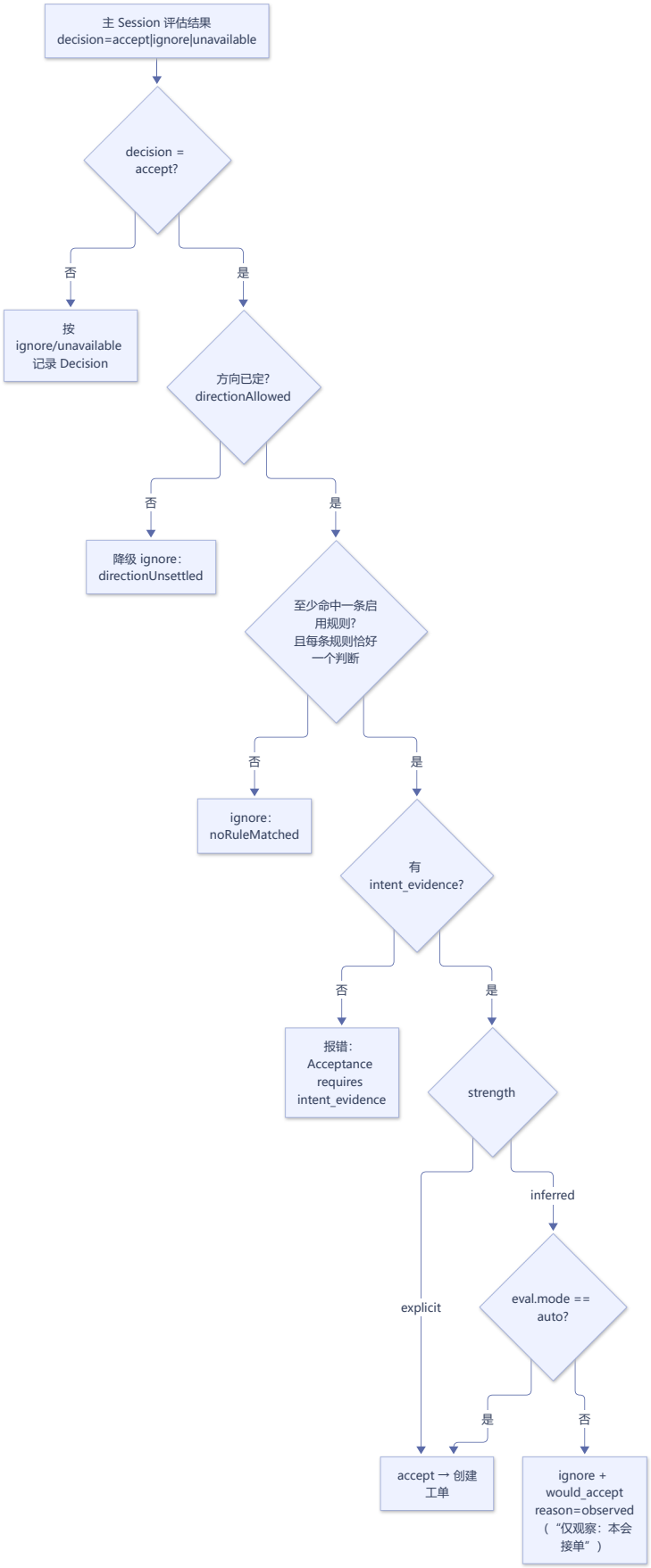
5. 决策：接不接、以什么强度接

5.1 准入 gate（先后顺序）



`Mandate.basis` 会记录 `source / scope / row / sender / rules / via`，用于后续**重新验证**（Policy 变化、工单恢复）。

5.2 评估换成 outcome



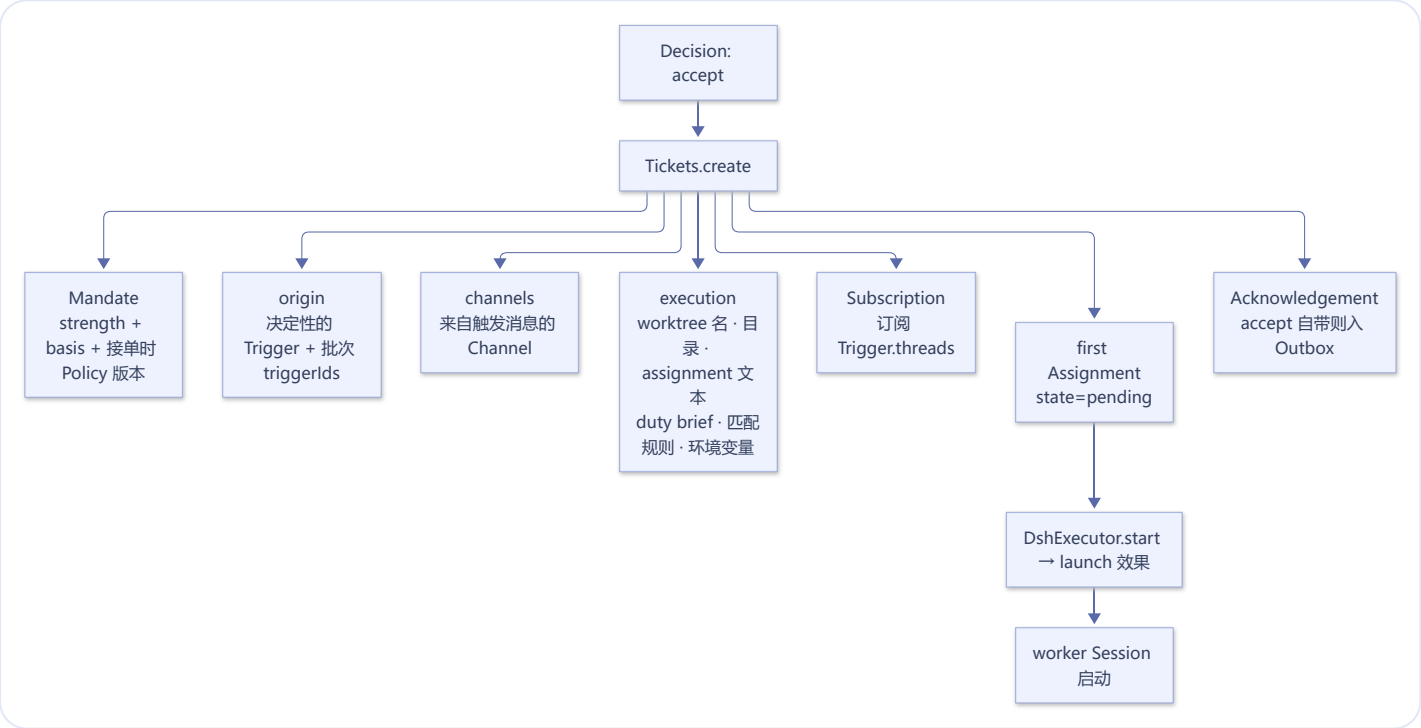
关键语义（常见误解）：`off / observe / auto` 只决定**主动候选（inferred）**是否接单； **明确请求（explicit）在任何模式下都会按派工规则路由**（`src/policy.ts:55`、`src/app/prompts.ts:45`）。 因此“仅观察”下，看板卡进监听列仍会被接单——这是设计行为，不是漏拦。

5.3 常见 Decision reason

reason	含义
`explicitRouted`	明确请求，直接路由接单（Kanban 卡进监听列常见）
`observed`	观察模式：本会接单，记为 `would_accept`
`startedTicket`	该触发已经开过工单，不能重复接
`timeout`	评估超时（`unavailable`）
`policyChanged`	评估期间 Policy 变了，重新评估/作废
`noRuleMatched` / `directionUnsettled`	规则未命中 / 方向未定 → 观察
`not_a_member` / `no_mention` / `admission_ignored`	来源过滤（面板可见）

---

6. 接单：工单、授权、派工

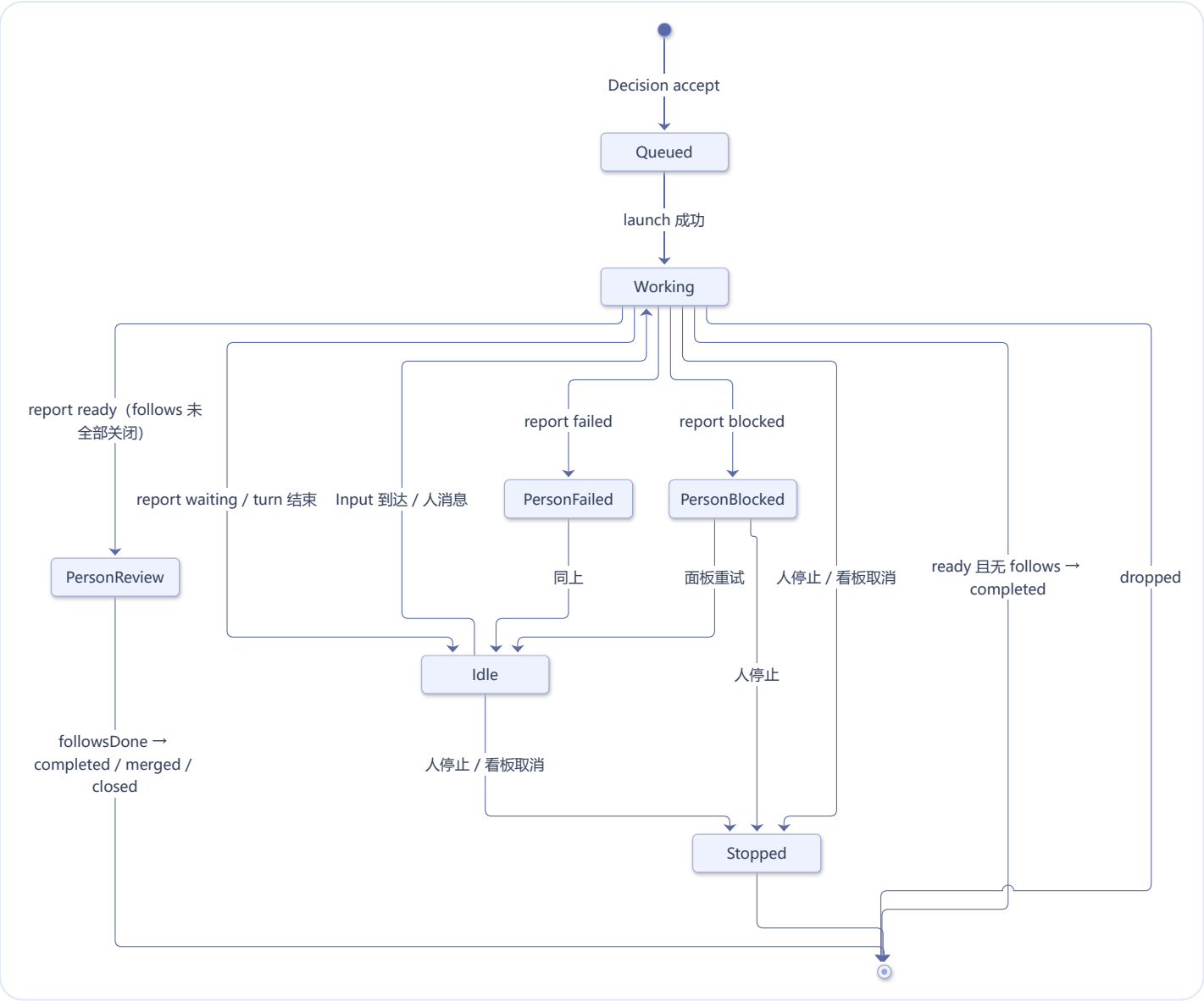


工单阶段（`ticketStage`，docs/003）：

阶段	条件
`person:delivery`	最近一次 Input 投递失败（`ticket.delivery`）
`person:answer`	有 `attention` 问题
`person:review`	最新 report `ready`（等人验收/关闭）
`person:blocked` / `person:failed`	最新 report 是 blocked/failed 且 **仍没有人**的消息
`working:queued`	Assignment `pending`
`idle`	report `waiting`，或 Assignment 已结束
`working`	其余（有运行中的 Assignment）

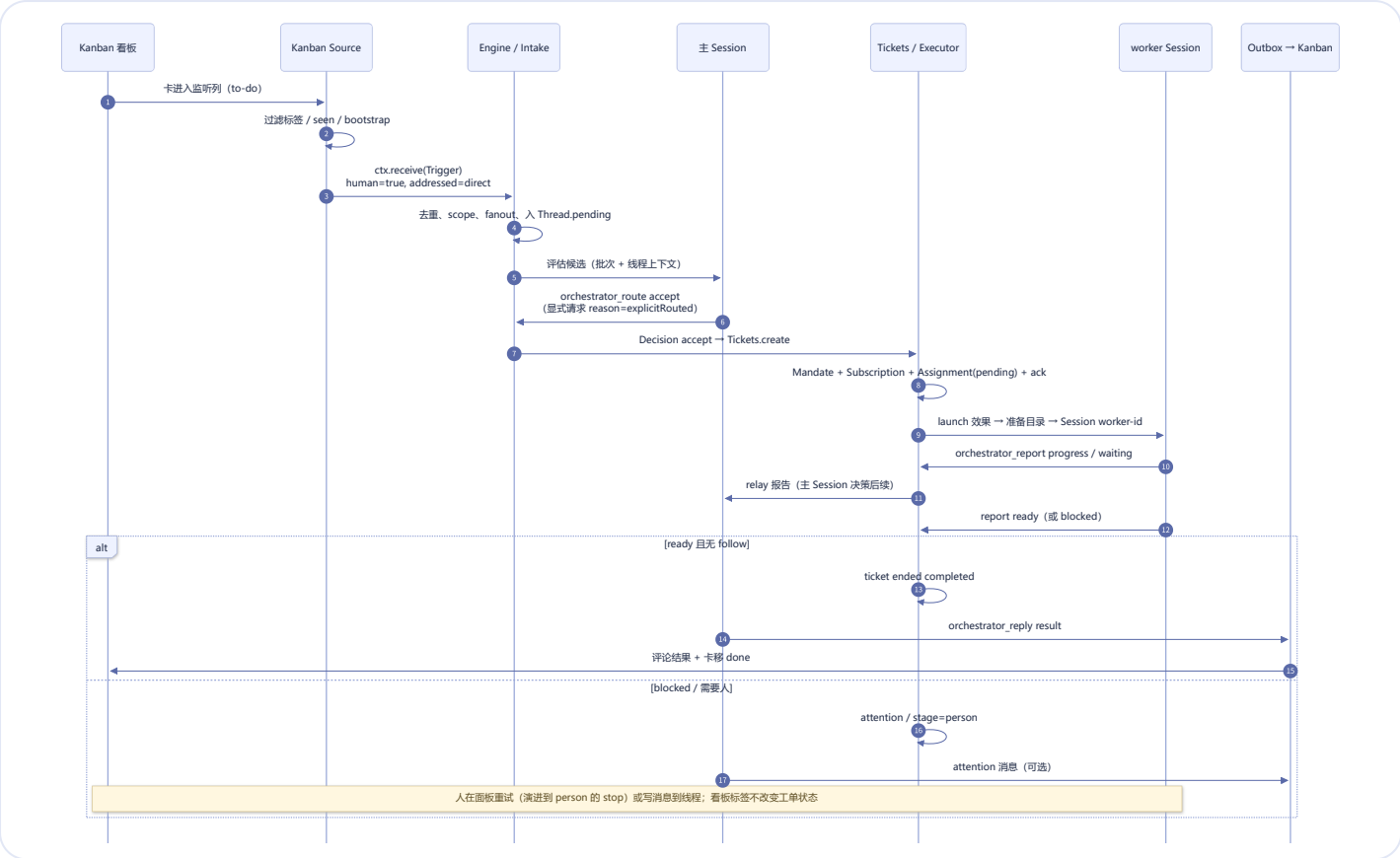
阶段	条件
`ended`	有 `ended`

工单生命周期：



---

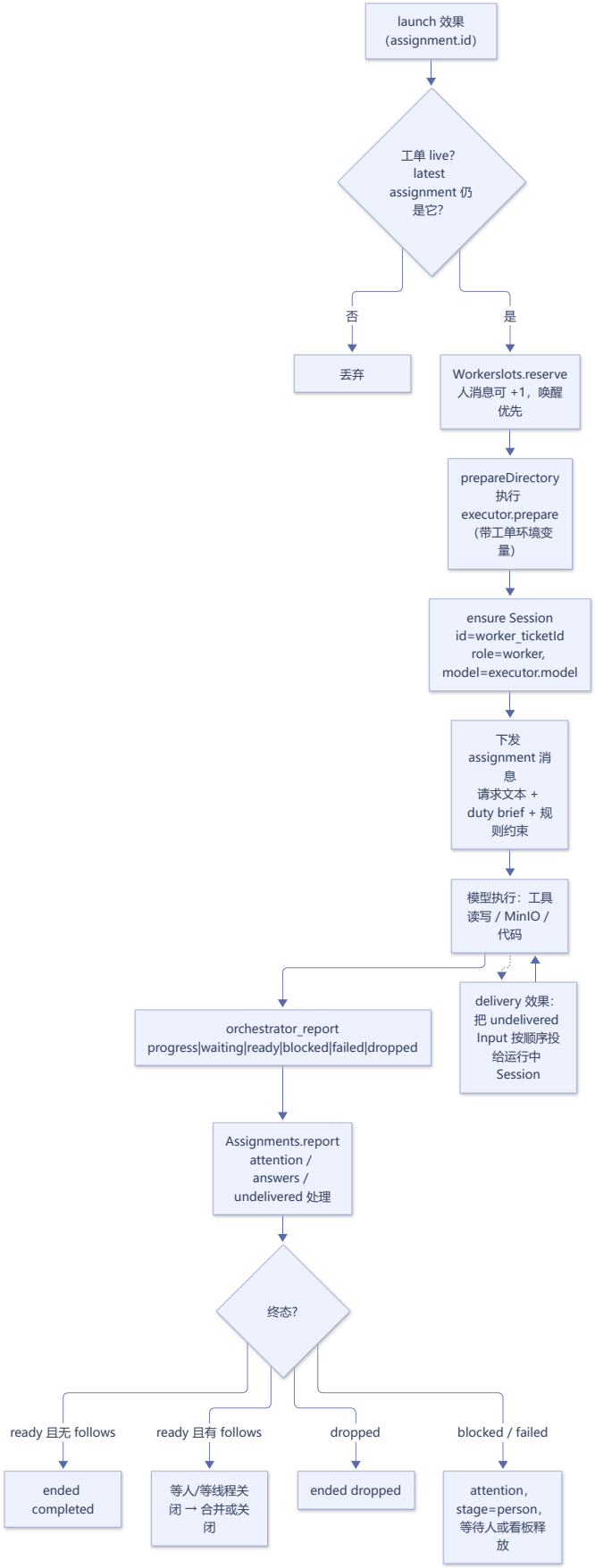
7. 看板卡的全链路时序（显式请求）



同一张卡在**运行中**被修改（评论/描述/列）时：`observeCards` 生成更新 Trigger → Intake → 该工单的 Input → `deliver` 效果 → worker 在同一条对话里读到，不新开工单。

---

8. 执行内部（DshExecutor）

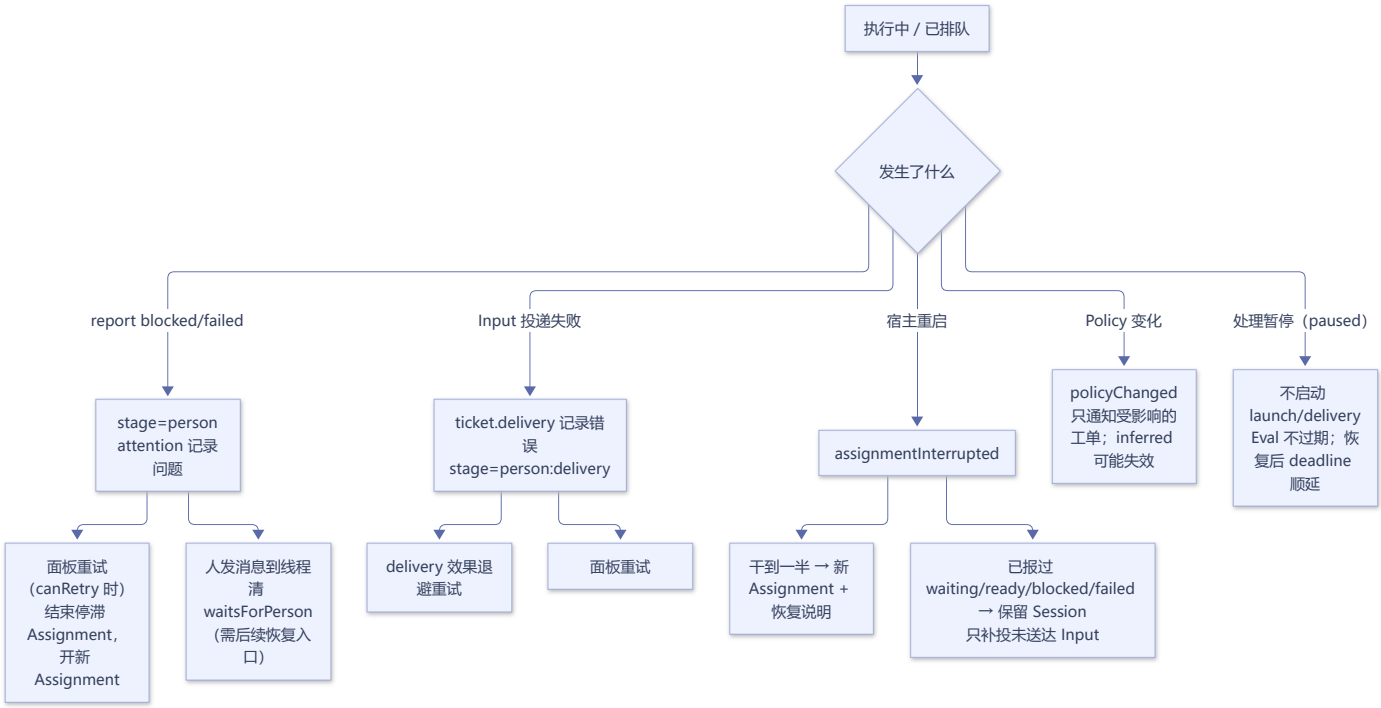


效果（`src/app/effects.ts`）关键语义：

效果	作用	失败行为
`assessment`	把候选卡送进主 Session 并通知“已送达”	2s 起指数退避

效果	作用	失败行为
`launch`	准备目录、建 Session、发 assignment	模型/网络瞬时错误重试 3 次；prepare 失败直接判失败
`delivery`	把工单的待投 Input 交给运行中 Session	2s 起退避；记录在 `ticket.delivery`
`relay`	worker 报告/问题、人的 follow-up 送入主 Session	保序，失败阻塞后续同类
`notice`	不唤醒主 Session 的提示	保序
`reply`	Outbox 发送到来源 Channel	5s 起退避；前面同会话消息失败会阻塞

9. 失败、等待与恢复



> **注意**：blocked/failed 工单收到人的消息后，`waitsForPerson` 变 false，但当前实现**不会**因此自动起新 Assignment； 受支持的恢复入口是**面板重试**。这条边界值得后续改造成“人类消息即恢复”。

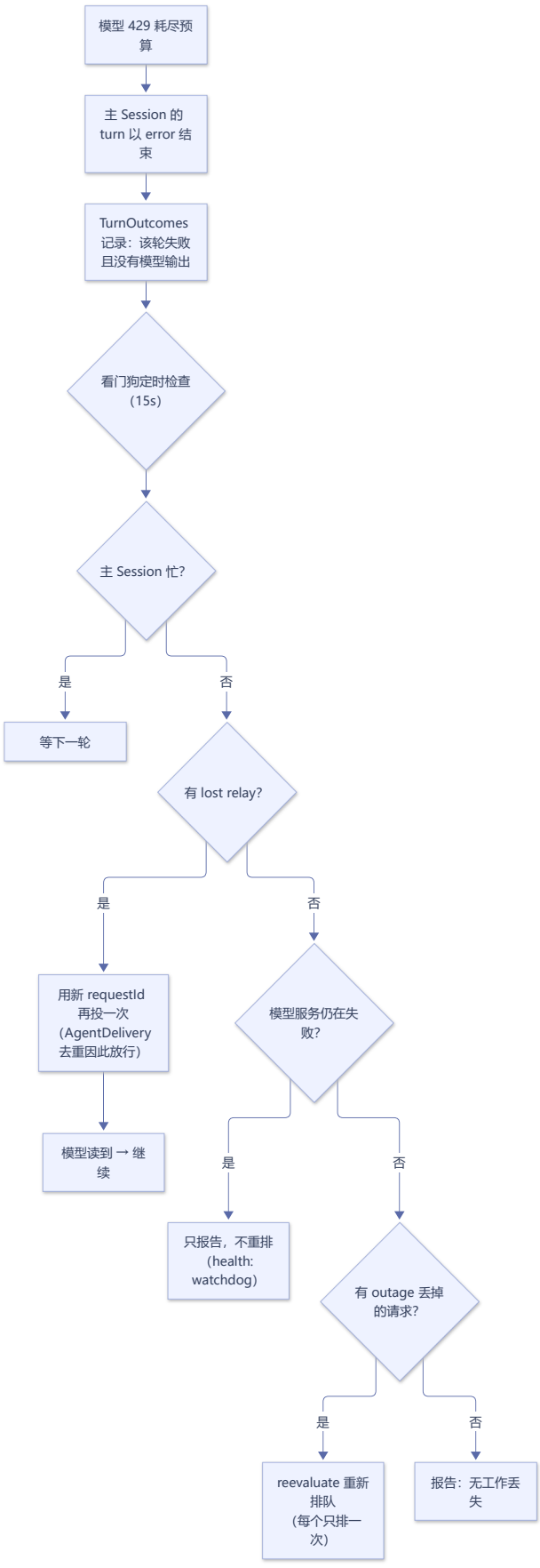
9.1 模型服务故障：主 Session 停摆与恢复（看门狗）

模型服务故障（429 预算耗尽、网关不可用）会让**主 Session 的那一轮 turn 以 `error` 结束**。此时有两处工作被拿走，且没有任何机制会把它们放回来：

丢失的东西	原来的后果	现在的恢复方式
relay（工人报告、follow-up、评估候选卡）	已投递但模型没读到；effect 行已删除 → **永久丢失**	看门狗按新的 requestId **再投一次**（`Sessions.redeliver`），模型读到为止
评估（候选卡无人判定）	Eval 过期 3 次后记为 `unavailable` → **请求被丢弃**	模型恢复后，看门狗用 `Engine.reevaluate` **重新排队**，再判一次

判定依据（`src/app/turn-outcomes.ts`）：

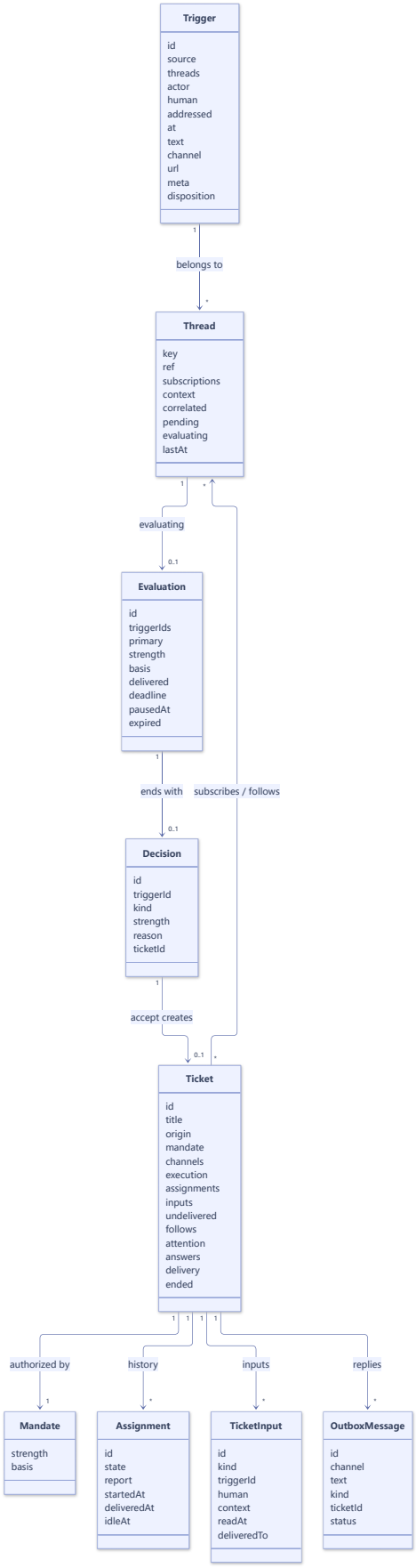
- `agent/inbox/claimed` 告诉我们**哪一轮取走了哪条消息**；
- `turn/end` 的 `reason.kind`（`error` / `aborted`）说明那一轮**失败**；
- 该轮**没有** **`assistant/message`**（`spoke === false`）说明模型**根本没读到**那条消息 → 判定为 lost。



> 面板上的 `watchdog` 健康条目会说明当前状态：
> `The model service is not answering; the main Session waits for it`、
> `The main Session's turn failed: N relay(s) sent again` 或
> `N request(s) the outage dropped are judged again`。 它的价值在于**把静默变成可见**：以前 429 之后面板什么也不显示。

---

10. 数据模型



关键字段的用途：

- **`Trigger.disposition`**：记录了它到达了哪些工单、是否被过滤、被谁退还、被哪些工单认领。
- **`Thread.subscriptions`**：之后的触发会 fan-out 给这些工单；`context` 列出“只读上下文”的工单。
- **`Mandate.basis`**：`source/scope/row/sender/rules/via`，Policy 变化或工单复活时据此重新验证。

- `Ticket.origin`：开单的 Trigger 与批次；`execution` 决定 worker 目录与 assignment 文本；`follows` 决定 `ready` 何时真正结束工单。
- `Assignment.report`：最新报告（state/summary/at），`waitsForPerson` 与 `ticketStage` 都读它。
- `TicketInput.readAt/deliveredTo`：谁在什么时候把哪条输入交给 worker，用于面板与恢复。

---

11. 关键不变量与常见误解

#	不变量 / 事实	代码位置
1	同一 Trigger 只存一次（幂等）；Kanban 同一任务只产生一次 Trigger	`core/intake.ts`、`KV.kaneo.seen`
2	一个 Thread 同时只有一个 Eval；批次内触发共享强度与决策	`core/evals.ts`
3	一个 Trigger 最多开一个工单（`startedTicket`）	`core/evals.ts`
4	每条启用规则必须恰好一个判断，accept 必须有 `intent_evidence`	`policy.ts` `assessmentOutcome`
5	`off/observe/auto` 只约束 **inferred**；**explicit 在任何模式都接单**	`policy.ts:55`、`app/prompts.ts:45`
6	`ready` 结束工单的前提是它跟随的 Thread 都已关闭；否则进入 `person:review`	`core/assignments.ts`、`core/tickets.ts` `followsDone`
7	只有人写的消息能复活已结束工单；`cancel`/`stop` 不复活	`core/tickets.ts` `wake`
8	看板观察的更新走 Input，不新开工单；看板的标签与列变化只作为更新来源，不改变工单状态	`sources/kaneo.ts`、`core/tickets.ts` `addInput`
9	所有对外回复走 Outbox（幂等 id + 退避重试），失败会显示在面板	`core/replies.ts`、`app/outbox.ts`
10	面板的“仅观察”不是“全局暂停”；要全局停做用暂停/关来源/清监听列	`policy.ts`、`docs/005-panel.md`

---

12. 从代码读流程的入口索引

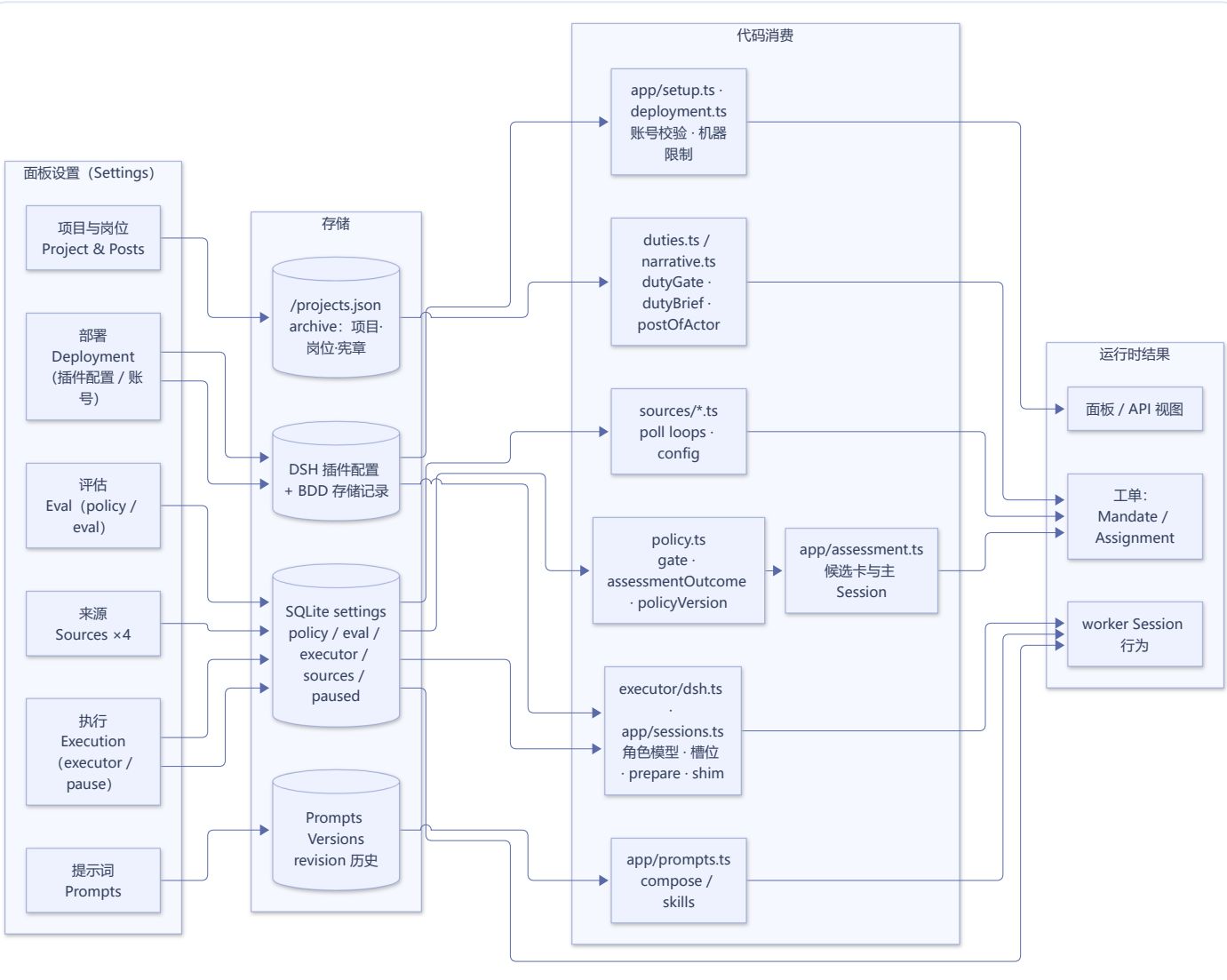
环节	入口
来源轮询与触发	`src/sources/kaneo.ts` `pollRound` / `kaneoTrigger`；`src/sources/ntfy.ts`、`src/sources/gitea.ts`、`src/sources/s3.ts`
摄入与扇出	`src/core/intake.ts` `Intake.receive`
轮次调度	`src/app/passes.ts` `Passes.pump`、`src/core/engine.ts` `evaluateDue`
评估批次与超时	`src/core/evals.ts` `start` / `due` / `delivered` / `decide`
候选送达与评估	`src/app/assessment.ts`、`src/app/tools/main.ts`（`orchestrator_eval`、`orchestrator_route`）
准入与判定	`src/policy.ts` `gate`、`assessmentOutcome`、`revalidate`
工单与授权	`src/core/tickets.ts` `create` / `addInput` / `resume` / `retry` / `stop`
执行	`src/executor/dsh.ts` `start` / `deliver` / `launchQueued` / `prepare`
报告与阶段	`src/core/assignments.ts` `report`、`src/core/stage.ts` `ticketStage`
回复与回写	`src/core/replies.ts`、`src/app/outbox.ts`、`src/sources/kaneo-channel.ts`

环节	入口
观察/阻塞/取消	`src/sources/kaneo.ts` `observeCards`

13. UI 设置 → 存储 → 代码 → 运行时：映射与关联

> 本节以 **2026-11-11 08:45 (+08) 的线上实际设置** 为样本，字段名取自 `GET /orchestrator/api/settings` 与面板表单， 代码位置是实际读取该值的地方。目的：解释“面板上改一个值，究竟改变了哪段逻辑”。

13.1 一图看全



13.2 面板分组总览

面板分组	存储位置	保存入口 / 校验	主要消费代码	影响什么
项目与岗位	`<stateDir>/projects.json` (archive)	`POST narrative` / `POST narrative/charter`;	`narrative.ts`、`duties.ts`、	岗位身份（actor）、职责

面板分组	存储位置	保存入口 / 校验	主要消费代码	影响什么
		`ArchiveSchema` 校验，保存时把 `kaneoUser` 映射为 `kaneoUserId`	`kaneo-users.ts`、`enrichTicket`	门禁、duty brief、WIP 展示
来源 ×4	`settings.sources[<id>]`（enabled + config）	`PUT settings/source:<id>`；各 `*ConfigSchema` 校验；改 `restarts` 字段会重启插件	`kaneo.ts` / `gitea.ts` / `ntfy.ts` / `s3.ts`	轮询、触发、观察、hold、取消、健康检查
评估	`settings.policy`、`settings.eval`	`PUT settings/policy`、`PUT settings/eval`	`policy.ts`、`assessment.ts`、`prompts.ts`	准入 gate、规则匹配、接单模式、评估超时、Policy 版本
执行	`settings.executor`、`settings.paused`（独立块）	`PUT settings/executor`、`POST executor/pause`；`executor` 的环境字段仅 `panel` 可改	`executor/dsh.ts`、`sessions.ts`、`setup`（命令 shim）	角色模型、并发、工单目录、prepare、禁命令、暂停
提示词	Prompts Versions（revision 历史，`KV.prompts.*`）	`POST prompts`，带 base revision 乐观锁	`prompts.ts`、`sessions.ts`	主 Session / worker 的系统提示词与 skill
部署	DSH 插件配置（machine）+ BDD 存储记录	插件 Config 表单；保存后 `SETUP_FIELDS` 触发校验/重启	`deployment.ts`、`setup.ts`、`bdd-storage.ts`	机器限制、账号解析、setup 健康

13.3 项目与岗位（archive）

线上值（`GET narrative`）：

项目	postId	角色	kaneoUserId	WIP	watchColumns	duties
Superagent（`at8xrho45bisco2umjzgl1xp`）	`ux-fdep`	worker	`yO6O7g…VvE`	3	`to-do`	空
同上	`api-fdep`	worker	`7MaRbn…cbZ`	3	`to-do`	空
同上	`pm-rsi`	pm	`bsFdwg…J3zG`	2	`to-do`	空

UI 字段	存储字段	实际代码	运行时作用
项目 id / 名称 / slug	`Project.id/name/slug`	`projects.ts`、`charter.ts`	看板项目与宪章归属；与 Kanban `projectId` 对应
宪章正文 / 发布	`Project.charter.text/draft/history/version`	`readCharter`、`charter.ts`；`dutyBrief` 带 `Charter vN`	提示词里的宪章、duty 的 `charterFocus`；发布版被职责 pin
岗位 id / 名称 / 角色	`ArchivePost.postId/name/role`	`posts.tsx`、`enrichTicket`	`pm` 岗位走 PM 执行门禁；worker 岗位接 duty
Kanban 账号 / 用户 id	`kaneoUser` → `kaneoUserId`	`kaneo-users.ts` 保存时按看板成员映射	**actor 匹配**：`narrative.ts:167 postOfActor(actorId)` 决定这条卡属于哪个岗位
Gitea 账号 / ntfy 发送者	`giteaUser` / `ntfySender`	仅 archive 保存与展示	岗位身份说明；不参与 gate（gate 看来源策略）
WIP	`wipLimit`	只在 `posts.tsx` / `overview.tsx` 展示	**不限制并发**；真正并发上限是 `executor.maxWorkers`

UI 字段	存储字段	实际代码	运行时作用
instance	`instance`	岗位视图展示（多实例预留）	目前为空 = 未部署
watchColumns	`watchColumns`	岗位视图展示	不覆盖 Kanban Source 的 `watchColumns`（来源级才生效）
职责下拉	`duties[]`	`dutyTakenBy` / `dutyGate`	空则回退到职责文档里的“承接岗位”行
职责标签	卡片 `duty:<id>`	`dutyGate`	未知职责/非本岗职责 → 拒绝接单；已知职

> 关联：`dutyGate` 在 **评估阶段**（`runtime.ts` 的 `duty` 钩子 → `assessment.ts`）影响候选；`dutyBrief` 在 **派工阶段**进入 assignment 文本。

13.4 来源（四个卡片 + 每来源策略）

每来源策略（`settings.policy.sources.<id>`）是 gate 的输入：`allowed`（能发起任务的人）、`admission`（观察全部消息时的准入表）。 线上：`kaneo.allowed=[吴志平(bsFd…)]`、`ntfy.allowed=[wu.zhiping]`、`gitea.allowed=[shawoo]`，三者的 `admission` 都为空。

Kanban 看板（`settings.sources.kaneo.config`，rev 82）

表单字段	线上值	代码读取处	作用
服务地址 / API key	`https://pl.feg.cn` / 空（用 BDD 记录）	`kaneoAccountOf`、`storedKaneoAccount`	客户端连接；空 key 时来源 paused
项目 / 工作区	`at8xrho45bisco2umjzgl1xp` / `lDarCeU…`	`columnTasks`、`policyWorkspace`	读哪块板；策略成员名单按工作区
监听列 `watchColumns`	`to-do`	`watchColumnsOf` → `pollRound`	**决定哪些卡会变成 Trigger**
必需标签 `requiredLabels`	空	`requiredLabelsOf`	只接带这些标签的卡
首次轮询 `bootstrap`	`skip`	`KV.kaneo.bootstrap`	首次只记基线不接历史卡
认领 / 交付列	`in-progress` / `done`	`claimColumnOf`、`doneColumnOf`	工单开始时/交付后移列
观察更新 `observeUpdates`	`true`	`observeCards`	运行中改评论/描述/列 → 作为 Input
其它来源先入库 `ingestEvents`	`true`	`kaneo-inbox.ts ingestEvents`	ntfy/Gitea 事件先建卡，而不是直接触发
轮询间隔 / 失败阈值 / 每列上限	`65s` / `3` / `50`	`pollRound`、watch loop	轮询节奏、健康报错、单次读取量
无负责人时 actor	`kaneo`	`kaneoTrigger`	卡片没有负责人/创建者时的身份回退
每来源策略	见上	`policy.gate`	谁把卡放进监听列才算合法请求

Gitea（宪章仓库，rev 86）

表单字段	线上值	代码读取处	作用
服务器 / 邮箱 / 密码 / token	`https://git.feg.cn` / `wu.zhiping@feg.cn` / 空 / 空	`GiteaClient`	匿名可读仓库即可；有账号则先校验登录
宪章仓库	`shawoo/rsi_superagent`	`verifyGitea`（保存校验）、`charterHealth`、`readCharter`	宪章与 `duties/` 的唯一来源
宪章文件	`README.md`	`charterPath`	宪章入口文件；`duties/` 在它旁边
—（隐藏旧字段）	`repository=shawoo/PDP`、`listen=repo`、`observeIssues…`	仅在旧配置里保留	**当前构建不轮询 Issue/PR**：`apply()` 只注册 kind + `charterHealth`，`watch()` 未启动；这些值不产生任务

ntfy（rev 112）

表单字段	线上值	代码读取处	作用
服务器 / 主题	`https://msn.feg.cn` / `pub_at8…`	`NtfyClient`	读主题历史
认证方式 / 账号 / 密码 / token	`none` / 空	`ntfyAccountOf`	私有主题鉴权
发送者 / agent	`#dsh-superagent` / `dsh-superagent`	`channel.send`、`ntfySenderOf`	回写消息标题与发送者约定
标签过滤 `ingestTags`	`dsh`	`pollRound`	只处理带该标签的消息（宿主自己的回复会被过滤）
首次轮询 / 轮询间隔 / 每次条数	`skip` / `65s` / `100`	`pollRound`	历史基线、节奏、单次读取量
与 Kanban 联动	`ingestEvents=true`	`kaneo-inbox.ts`	消息先建成 Kanban 卡（`to-do`），由看板调度；Kanban 的 `ingestEvents` 关掉时 ntfy 才会直接产生 Trigger

MinIO（rev 1）

表单字段	线上值	代码读取处	作用
服务地址 / 桶 / access / secret / region / prefix	全空	`s3AccountOf` + `storedConnections`	空则用 BDD 存储记录 `/BDD/API/AI/dsh/storage/s3`；仅探活与 worker 工具
巡检间隔 / 失败阈值	`305s` / `3`	watch loop	健康检查节奏
生产任务	—	—	**不产生 Trigger**，只做资产读写

13.5 评估（policy + eval）

UI 字段	存储	线上值	代码	运行时作用
接单模式	`settings.eval.mode`	`observe`	`assessmentOutcome`、`gate`、`prompts.policySection`	只拦 **inferred**；explicit 仍接单（§5.2）

UI 字段	存储	线上值	代码	运行时作用
评估超时	`settings.eval.timeoutSeconds`	`120`	`settingsPolicy` → `evalTimeoutMs`；`Evals.delivered/due`	candidate 送达后 120s 未决策 → 过期重排，3 次后 `unavailable`
研判模型	`settings.eval.model`	`null`（用默认）	`Sessions.ensure` role=orchestrator	主 Session 的模型
接单规则	`settings.policy.rules`	copy + visual，均启用	`assessmentOutcome`、主 Session 提示词	每条规则必须恰好一个判断；命中且有 `intent_evidence` 才可接
各来源策略	`settings.policy.sources.<id>`	allowed 3 人；admission 空	`policy.gate`	谁能发起、未 @ 时按准入表定 explicit/inferred/filter
策略版本	由 rules + mode + sources policy 计算	`policyVersion`	`policyVersion()`、`Tickets.create` 记录、`policyChanged`	变更会通知受影响工单；工单记下接单时版本

> 这解释了此前的现象：线上 `mode=observe`，但 Kanban 卡进入监听列是 **explicit**，因此决策 `explicitRouted` 仍会接单。

13.6 执行（executor + pause + 环境变量）

UI 字段	存储	线上值	代码	运行时作用
worker 模型	`executor.model`	`aife / MiniMax-M3.1-Flash`	`Sessions.ensure` role=worker	worker Session 用哪个型
最大并发	`executor.maxWorkers`	`1`	`DshExecutor` `WorkerSlots`	同时运行的 worker 数上限（人消息可临时 +1）
执行目录	`executor.directory`	空	`ticketExecution`	空 → `<stateDir>/work`；面显示 `environment.roo`
prepare 命令	`executor.prepare`	空	`prepareDirectory`	新工单目录准备命令，可用 `DSH_SUPERAGENT_*` 量
禁止命令	`executor.blockedCommands`	空	`setup.blockCommands`（重写 worker-bin shim）+ `DSH_SUPERAGENT_BLOCKED_COMMANDS`	shell 层物理拦截；改后即重建 shim
暂停	`settings.paused`（独立块）	`false`	`Passes`、`DshExecutor`、`Outbox`、`Evals.due`	停 launch/delivery/rep与评估过期；**来源仍记录触发**
环境变量	由 `environment.variables` 派生	见 `GET settings.environment`	`execution-environment.ts`、`EnvironmentVariables` 选择器	插入 prepare 命令；工变量（DIR/NAME/TICKET..与来源变量

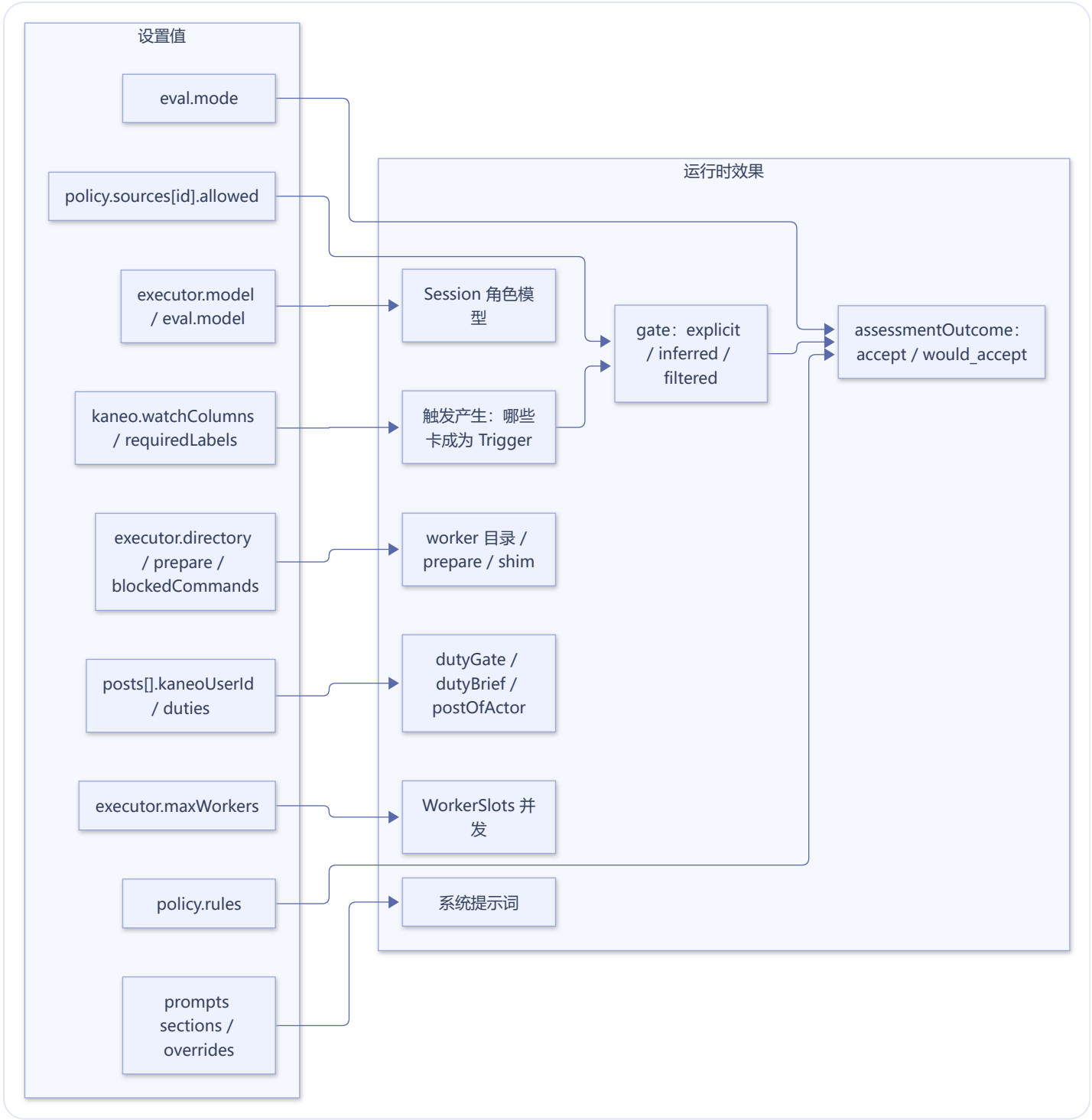
13.7 提示词

UI 字段	存储	代码	作用
三个团队段落（沟通 / 工单范围 / 交付要求）	Prompts Versions 的 `communication/scope/delivery`	`prompts.ts` `TEAM_SECTIONS` → `compose`	主 Session 与 worker 共用的行为约束
skill 配置	`skills` map（名字 → 文本/路径）	`skills()` 注册 DSH skill	worker 技能可用性
内置段落覆盖	`overrides`（key → `{text, base}`）	`PART_KEYS`（runtime/、*skill/*）	替换 ①②③ 等段落；保存后通知主 Session

13.8 部署（DSH 插件配置 + 账号）

UI 字段	存储	线上值	代码	作用
stateDir	DSH 插件配置（machine）	`D:/…/dsh/state/superagent`	`machineOf`、`runtime`	状态库、projects.json worker 目录的根
promptMaxChars	同上	`100000`	`resources.ts`	提示词/上下文截断上限
admissionContextMaxPages / Timeout	同上	`10` / `15000ms`	`assessment.ts`	评估时读取线程/仓库上下文的页数与超时
GitHub / 飞书账号	CLI 解析	空	`resolveDeployment`、`resolveFeishuBot`	GitHub/飞书源是否可用
Kanban / Gitea / ntfy 账号	BDD 存储记录 `/BDD/API/AI/dsh/storage/*`	已配置	`storedConnections`、`setup.checkAccounts`	REST 来源的连接与保存校验 S3 同理

13.9 设置联动的关键链



13.10 “UI 有，但不按你想的那样生效”清单

现象	真实逻辑
「仅观察」仍接单	只拦 inferred；Kanban 卡是 explicit，任何模式都路由
岗位 WIP 填 3，仍可能只有 1 个在执行	WIP 仅展示；并发上限是 `executor.maxWorkers`（线上 = 1）
岗位里填了 watchColumns，看板却没变化	岗位字段仅展示；真正生效的是 Kanban 来源的 `watchColumns`
Gitea 卡片里有旧事件仓库/监听配置	当前只做宪章仓库；`apply()` 不启动事件 watch
MinIO 卡片全空却健康	空字段回退 BDD 存储记录 `/BDD/API/AI/dsh/storage/s3`
改 Gitea 宪章路径后插件重启	`charterPath` 在 `SETUP_FIELDS.gitea` 里，属重启字段
保存按钮有时灰/有时没有	有未保存改动才出现；`testable` 来源必须先测试通过才能保存

现象	真实逻辑
暂停后卡还在产生触发	暂停只停 launch/delivery/reply 与评估过期，来源照常记录
改策略后运行中工单行为变了	`policyVersion` 变化 → `policyChanged` 通知受影响的工单
面板提示“已保存”但 revision 没变	值经 trim/规范化后与旧值相同，视为无变化、不落库

本文只描述当前实现；图中“人/PM”指在 Kanban 或面板上操作的人。修改流程语义（例如让 observe 也拦明确请求）时，请同时更新本文件与 docs/007-architecture.md。

由 FLOW.md 生成：Markdown → HTML（AIOS 同款深色主题），Mermaid 图在无头 Chrome 中预渲染为内联 SVG；PDF 由 scripts/html-to-pdf.mjs 打印。